# Open Source Software - Audit

**Course**: Open Source Software

**Type** : Project Report

**Name** : Rohit Dubey

**Registration Number** : 24BAI10353

**University:** VIT Bhopal

**Slot** : F11

**Faculty:** Dr. Saravanan D

**Submission Date** : 31st March 2026

# 1. Introduction

The landscape of modern technology has undergone a fundamental change; software today is really the product of a single company working behind locked doors. Instead, the most resilient and innovative tools are built through a model of shared intelligence where code is improved and refined by a global network of contributors. This is the essence of the open-source concept: by allowing anyone to access, audit, and modify the underlying source code, open source fosters an environment of transparency that proprietary models simply cannot. It transforms software from a static product into a living project that evolves based on the collective needs of its users rather than the profit motives of a single board of directors.

The importance of this model cannot be overstated. As the digital infrastructure we use every day—from the servers powering the Internet to mobile applications to the servers that we use for our phones—relies heavily on open-source foundations. Most developers today utilize open-source libraries and frameworks almost unknowingly, standing The shoulders of thousands of volunteers who came before them.

For this report, I have chosen to analyze VLC Media Player, a project that perfectly embodies the practical success of open-source philosophy. Developed by the Video LAN project, VLC is a highly versatile, cross-platform multimedia player capable of handling almost any file format without the need for external codecs. I choose VLC because of its ubiquity; it is one of the few pieces of open-source software that has achieved household name status. It serves as a perfect case study because it isn't just a niche tool for developers; it is a reliable, high-performance application used by millions of non-technical people worldwide, proving that open-source software can compete with and often outperform commercial giants.

# 2. Part - A The origin story of VLC

## 2.1 the digital tower babel; Life before VLC

To understand why VLC matters, one has to remember the frustration of the digital media landscape in the late 1990s and early 2000s. Back then, playing video was not a

plug-and-play experience. It was a technical minefield. Most media players were pro-retailing and tethered to specific ecosystems. If you had to file in a certain format,, you needed specific codec. Translate it, and those codecs were often tucked behind paywalls or restricted by licensing fees. Users were frequently met with the dreaded "Format not supported" error message. Further, the idea of streaming high-quality video over a local network was nearly impossible for the average person. The software of that era was designed to be a gatekeeper rather than a gateway, forcing users into a cycle of downloading endless plugins and paying for premium pro versions of players that still felt limited.

## 2.2 the video land rebellion:  innovation in the dorms

VLC did not begin in a corporate boardroom with a multimillion-dollar budget; it began in a dorm room in Paris in 1996. It was a video LAN project. It was driven by a group of students who were tired of technical restrictions of their time. These students weren't looking to build a commercial product to dominate a market; they were looking for a practical solution to a very specific problem. They wanted AMPEG 2 videos across the universe's high-speed network. This academic environment was the perfect incubator because they weren't answering to shareholders or marketing department; they were free to experiment with radical ideas such as a building a player that carried its own brain, internalizing all the necessary codecs so the users would never have to hunt for them again

## 2.3 The Leap to Open Source: Breaking the university walls

By 2001 the project reached a crossroads. The students realize that if VLC remained a private university project, it would eventually wither away as they graduated. They made the strategic and ethical decision to release VLC under the GNU General Public License (GPL). This wasn't just a way to share code; it was a way to ensure the software's quality. They survived by making it open source; they invited the world to help them build it. They wanted flexibility to let anyone, whether a developer in Tokyo or a student in Mumbai, modify, improve, and port the software to new operating systems. This move stripped away the restrictions of proprietary software and allowed VLC to grow far beyond the walls of a single French university, evolving into a global standard for media playback.

# 3. License analysis—the rules of the game

## 3.1 Understanding the GPL: more than just a free sticker

VLC Media Player operates primarily under the GNU license, with certain libraries using the lesser GPL/LGPL. While many people think free software just means you don't have to pay for it, the license is actually a sophisticated legal framework that defines how the software can be used, modified, and shared. Unlike a standard end-user license agreement EULA from a company like Microsoft, which usually tells you what you cannot do, the GPL is designed to protect your rights; it acts as a constitution for the code, ensuring that the software remains open regardless of whose hand it passes through.

## 3.2 The four freedoms: soul of VLC

the VLC license, you have to look at the four freedoms defined by the Free Software Foundation, all of which VLC satisfies completely. First, the complete freedom to use VLC for any purpose, whether you are a student watching a lecture, a business playing a presentation, or a government agency without needing permission or paying a subscription. Secondly freedom to study Because the source code is public, anyone can look under the hood to see how VLC handles data or plays a specific video format. The freedom to modify means that if a developer finds a bug or wants to add a new feature like support for a 360-degree camera, they have the legal right to change the code.

## 3.3. The copyleft clause: can companies profit?

A common misconception is that because VLC is open source, companies cannot sell it. In reality, the GPL allows a company to modify VLC and even sell it for profit; however, there is a massive catch known as "copyleft." If a company modifies VLC and distributes that version, they must make their modified source code available to the public under the same GPL license. They cannot take the community's hard work, add one secret feature, and then lock it away as proprietary software. This rule prevents corporate hijacking; it ensures that any improvement made by a private entity eventually flows back into the public commons for everyone to use.

### 3.4 GPL versus MIT The strictness of freedom

When choosing a license, developers often debate between the GPL and more permissive licenses like MIT. The MIT license is like a "do whatever you want" permit. It allows a company to take open-source code, turn it into a private product, and never share their changes. The GPL is a reciprocal license; it says if you benefit from our work, you must contribute your improvements back. In my opinion, if I were starting a project like that, I would choose the GPL. While the MIT license is great for small libraries, proprietary major tools like Media Player need the protection of the GPL. Without it, a large tech giant could easily absorb VLC's technology into a paid operating system and kill the original community project. The GPL's strictness is what keeps the ecosystem fair.

# 4. The Ethics of Open Source

## 4.1 Open Source as a Philosophy of Knowledge

It's called "open source," which is far more than a technical methodology for writing code; it is a profound statement about the nature of human knowledge. The movement is built on the belief that information is most powerful and reflects the philosophy of radical transparency and collaboration that mirrors the scientific method: you publish your findings, you allow others to criticize them, and you move toward a better result. It challenges the traditional silo mentality of the 20th century, suggesting that the lone genius working in a basement is less effective than a global community of contributors working in the light.

## 4.2 The great debate should also be open.

The question of whether all software should be open source or not is one of the most polarizing topics in technology. There are compelling arguments on both sides. The Case for Universal Openness: Proponents argue that open source is the only way to ensure true digital literacy and ethical accountability. When a code is public, backdoors and vulnerabilities can be spotted by anyone, not just the company that wrote it. The case for proprietary interests, conversely, is that companies argue that they need to protect their individual intellectual property to justify the

massive financial risks they take. If a company spends billion developing a revolutionary algorithm, they argue that they have a right to keep it secret to ensure a return on the investment for their employees and shareholders There is also a that making security-critical code public provides a road map for malicious hackers

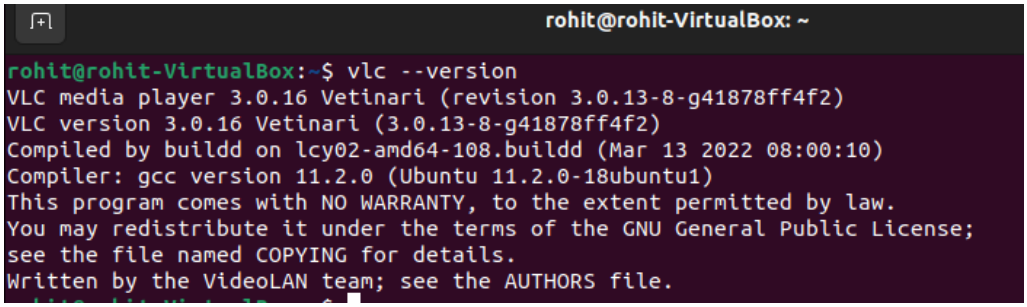
### 4.3 standing on the shoulders of giants

Standing on the shoulders of giants is the perfect summary of the open-source movement. In the modern era, no developer starts from zero. When I write a simple script today, I am using languages, libraries, and compilers that were built by thousands of people over decades. Open source accelerates development because it prevents us from reinventing the wheel every time we want to build something new. Some critics argue that it reduces originality as developers become copycasters of existing code. I, however, strongly believe it enhances innovation by automating the basic building blocks of software through open-source libraries. We free up human creativity to solve the next generation of problems; we aren't losing generality or originality; we are shifting our focus to higher-level challenges.

# 5. Part-B Linux Method

## 5.1 Installation Method

VLC was installed using the standard open Ubuntu package manager, apt, which is called the advanced package tool. This method is preferred over manual downloads because the package manager automatically handles dependencies, the smaller software libraries like VLC needs to run. By executing a simple command like "sudo apt install VLC," the system fetches the latest stable version from the official repositories, ensuring the software is authenticated and secure.



## 5.2 Binary and Key Directories

Once installed, the VLC files are distributed across specific system directories according to the Linux File System Hierarchy Standard.

- /usr/bin: contains the primary VLC executable that starts the application
- /usr/lib: stores the essential plugin libraries and codecs that give VLC its swiss army knife capabilities
- /usr/share: holds shared resources like icons interface translations and documentation available to users

```
rohit@rohit-VirtualBox:~$ dpkg -L vlc
/.
/usr
/usr/lib
/usr/lib/mime
/usr/lib/mime/packages
/usr/lib/mime/packages/vlc
/usr/share
/usr/share/applications
/usr/share/applications/vlc.desktop
/usr/share/bug
/usr/share/bug/vlc
/usr/share/bug/vlc/control
/usr/share/bug/vlc/presubj
/usr/share/doc
/usr/share/doc/vlc
/usr/share/doc/vlc/AUTHORS.gz
/usr/share/doc/vlc/README
/usr/share/doc/vlc/THANKS.gz
/usr/share/doc/vlc/copyright
/usr/share/lintian
/usr/share/lintian/overrides
/usr/share/lintian/overrides/vlc
/usr/share/metainfo
/usr/share/metainfo/vlc.appdata.xml
/usr/share/solid
/usr/share/solid/actions
/usr/share/solid/actions/vlc-openbd.desktop
/usr/share/solid/actions/vlc-opencda.desktop
/usr/share/solid/actions/vlc-opendvd.desktop
/usr/share/solid/actions/vlc-openvcd.desktop
/usr/share/doc/vlc/changelog.Debian.gz
rohit@rohit-VirtualBox:~$
```

```
rohit@rohit-VirtualBox:~$ which vlc
/usr/bin/vlc
```

```
rohit@rohit-VirtualBox:~$ ls -l /usr/bin/vlc
-rwxr-xr-x 1 root root 14640 Mar 13  2022 /usr/bin/vlc
```

## 5.2 Configuration and Data Files

While the application is in system right folders, user-specific settings are kept within the home directory. This allows multiple people on the same computer to have different VLC preferences.

- ~/.config/vlc: contains the VLCRC file which stores your custom settings such as interface language or hardware acceleration preferences
- ~/.local/share/vlc: stores user-specific data such as recently played media lists and cached thumbnails

```
rohit@rohit-VirtualBox:~$ ls -ld ~/.config/vlc
drwxrwxr-x 2 rohit rohit 4096 Mar 27 22:54 /home/rohit/.config/vlc
```

```
rohit@rohit-VirtualBox:~$ ls -ld ~/.local/share/vlc
drwx------ 2 rohit rohit 4096 Mar 27 22:54 /home/rohit/.local/share/vlc
```

## 5.4 user and permissions

For security reasons, VLC is designed to run under a standard user account, in this case Rohit, rather than the root or administrative user. This follows the principle of least privilege, ensuring that even if a malicious media file tries to exploit the player, it won't have the permissions to modify critical system files.

```
rohit@rohit-VirtualBox:~$ ps aux | grep vlc
rohit       1842  0.0  0.0   9080  2588 pts/0    S+   18:11   0:00 grep --color=auto vlc
```

## 5.5 Process and Execution

Because it starts, VLC operates as a standard user process. Unlike web servers or databases, it is not a demon, a service that starts with the computer; it only consumes resources, CPU, and RAM when the application is explicitly opened by the user and terminates those processes once the window is closed.

```
rohit@rohit-VirtualBox:~$ ps aux | grep vlc
rohit       1842  0.0  0.0   9080  2588 pts/0    S+   18:11   0:00 grep --color=auto vlc
```

## 5.6 Update Model

Updates for VLC are managed through the centralized app repository system. Whenever the video line community releases a security patch or a new feature, the package is updated in the Ubuntu repositories. When the user runs the system updates, `sudo apt upgrade`, VLC is automatically patched along with the rest of the system, ensuring the player stays secure against new vulnerabilities. VLC

# 6. Part 6 - FOSS ECOSYSTEM

## 6.1 A Web Of Dependencies

VLC is a masterpiece of integration, relying on open-source libraries to function. Most notably, it uses FMPG for decoding almost all audio and video formats alongside libraries like libMPEG2. VLC improves automatically, demonstrating the powerful network effect of open-source development.

## 6.2 Community-Driven Governance

Unlike projects controlled by big companies, VLC is maintained by VideoLAN, a nonprofit organization; its governance is entirely community-driven and meritocratic. This independence ensures that VLC remains free of advertisements and tracking and corporate agendas

## 6.3 Strategic Position in the Tech Stack

VLC is a critic of the Linux multimedia layer. Its core engine, libVLC, is used far beyond desktop-powering video transcoding, signage, and mobile apps and media players.

## 6.4 Reflection: The Strength of The Commons

My takeaway is that open-source projects are inherently code-dependent. A bug fix in a minor library can enhance a major tool like VLC. This ecosystem creates a circular economy of

knowledge where progress is shared. In my view, this is the most sustainable way to build technology, as it prevent any single company from becoming a point of failure for the rest of the community

# 7. Part-D Comparison

| Feature | VLC | Windows Media Player |
|---|---|---|
| Cost | Free | Pre-installed |
| Source Code | Open | Closed |
| Format Support | High | Limited |
| Customization | High | Low |
| Updates | Community | Microsoft |

# 8. Conclusion

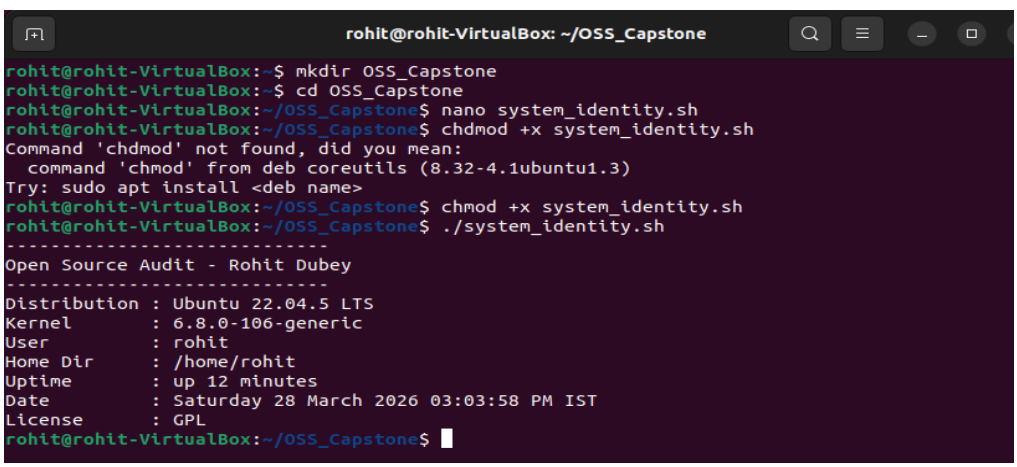
When comparing VLC Media to proprietary alternatives like Windows Media Player or QuickTime, it is important to be objective. For a casual who only watches standard file formats and prefer a minimalist Idget's work interface without ever opening settings menu a built-in player is a perfectly valid tool; however, these players often act as gatekeepers. They frequently lack support for older or niche formats and they are sometimes used as a vehicles for data collection or to push users towards paid ecosystem services

My final verdict: I would strongly recommend VLC Media Player as the primary media tool for any user, from students to professionals. The reason is simple: it prioritizes the user's intent over corporate strategy. While it may not have this leakage-modern UI, its Swiss Army knife capability to play virtually any file without needing extra codecs makes it technically superior.

# 9. Scripts

## 1. System Identity Report

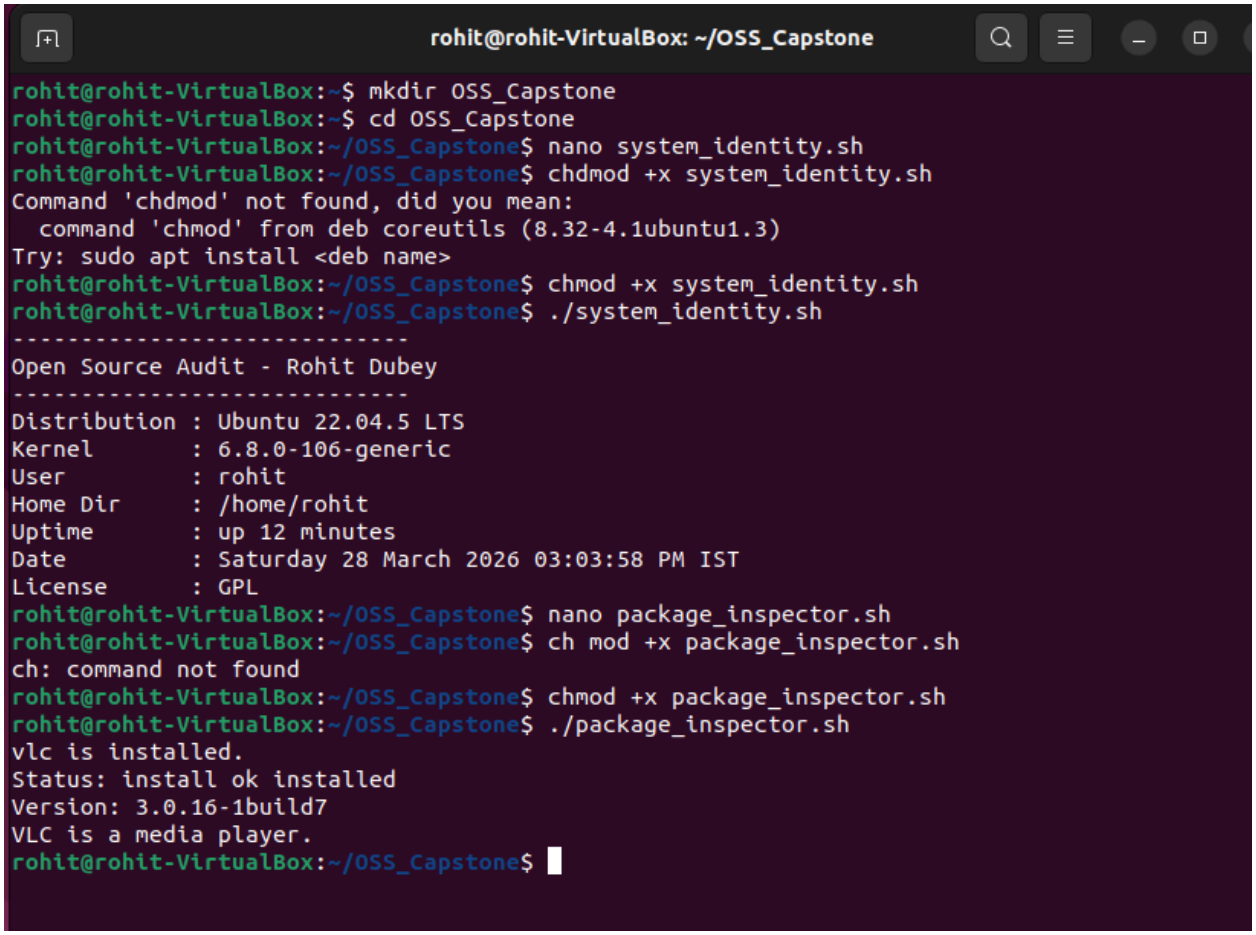
```
#!/bin/bash
# Script 1: System Identity Report
# Author: [Your Name] | Course: Open Source Software
# --- Variables ---
STUDENT_NAME="" # Fill in your name
SOFTWARE_CHOICE="" # Fill in your chosen software
# --- System info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
# --- Display ---
echo "================================="
echo " Open Source Audit — $STUDENT_NAME"
echo "================================="
echo "Kernel: $KERNEL"
echo "User: $USER_NAME"
echo "Uptime: $UPTIME"
# TODO: Add distro name, date, and license message
```




the script acts as a diagnostic tool that identifies the current environment by capturing the system's kernel version, active user, and session of time using Linux commands like `uname` and `uptime`. It stores these values variables and formats them into a clean, open-source audit report demonstrating how shell scripting can automate system information gathering

## 2. Foss Package Inspector

```
#!/bin/bash
# Script 2: FOSS Package Inspector
PACKAGE="" # e.g. httpd, mysql, vlc, firefox
# Check if package is installed
if rpm -q $PACKAGE &>/dev/null; then
echo "$PACKAGE is installed."
rpm -qi $PACKAGE | grep -E 'Version|License|Summary'
else
echo "$PACKAGE is NOT installed."
fi
# TODO: Add a case statement that prints a one-line
# philosophy note about the package based on its name
case $PACKAGE in
httpd) echo "Apache: the web server that built the open internet."
mysql) echo "MySQL: open source at the heart of millions of apps";
# TODO: Add your software and 3 others
Esac
```

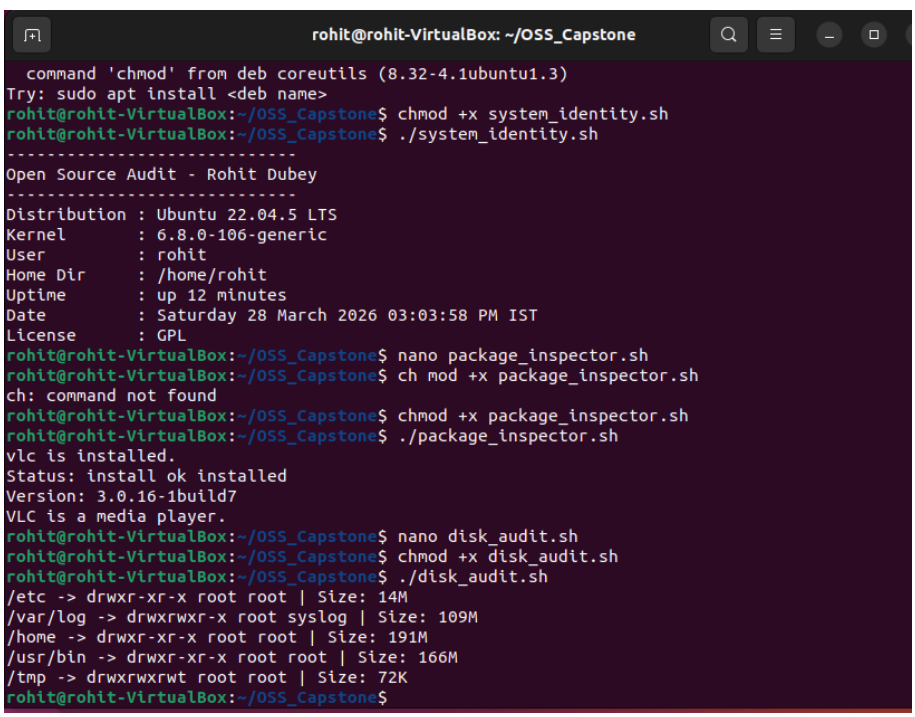



The script uses conditional logic to check if a specific software package is installed on the system using the RPM package manager. We found it extracts key metadata like the version and

license. It then uses a case statement to map the package name to a specific theological note, demonstrating how scripts can provide both technical data and a contextual information

### 3. Disk & Permission Auditor

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")
echo "Directory Audit Report"
echo "----------------------"
for DIR in "${DIRS[@]}"; do
if [ -d "$DIR" ]; ]; then
PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
echo "$DIR => Permissions: $PERMS | Size: $SIZE"
else
The Open Source Audit | VITyarthi Page 9
echo "$DIR does not exist on this system."
fi
done
# TODO: Add a section that checks if your software's
# config directory exists and prints its permissions
```

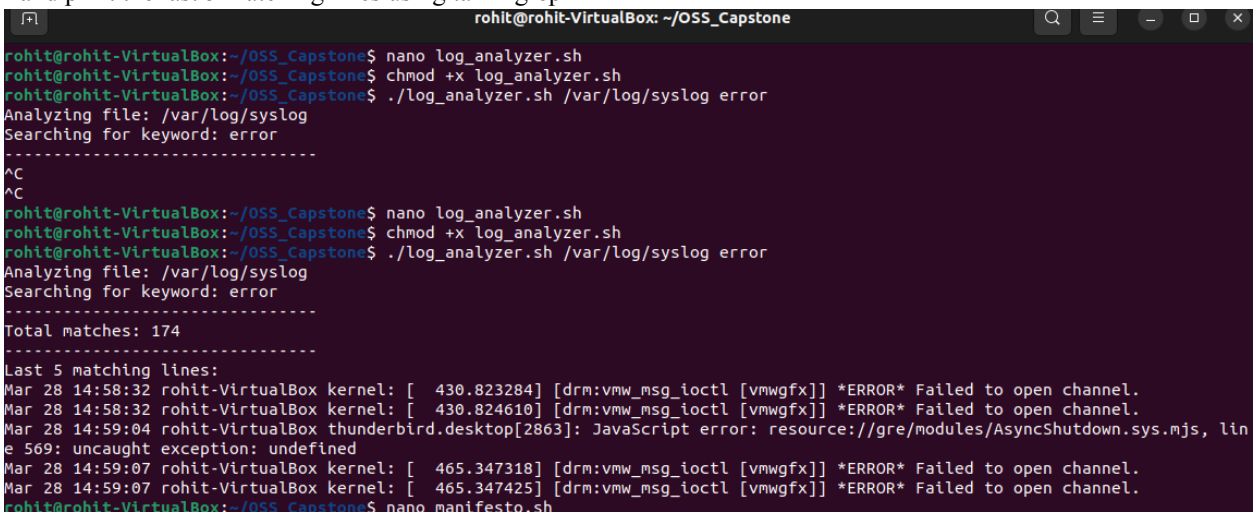



the script performs a system-wide audit by integrating through a critical array of critical directories using for a loop to verify their existence and retrieve their metadata. It employs

advance command tools like awk, du, and CUT to extract specific permission strings and disk sizes, providing a clear overview of the system storage and security questions.

### 4. Log File Analyzer

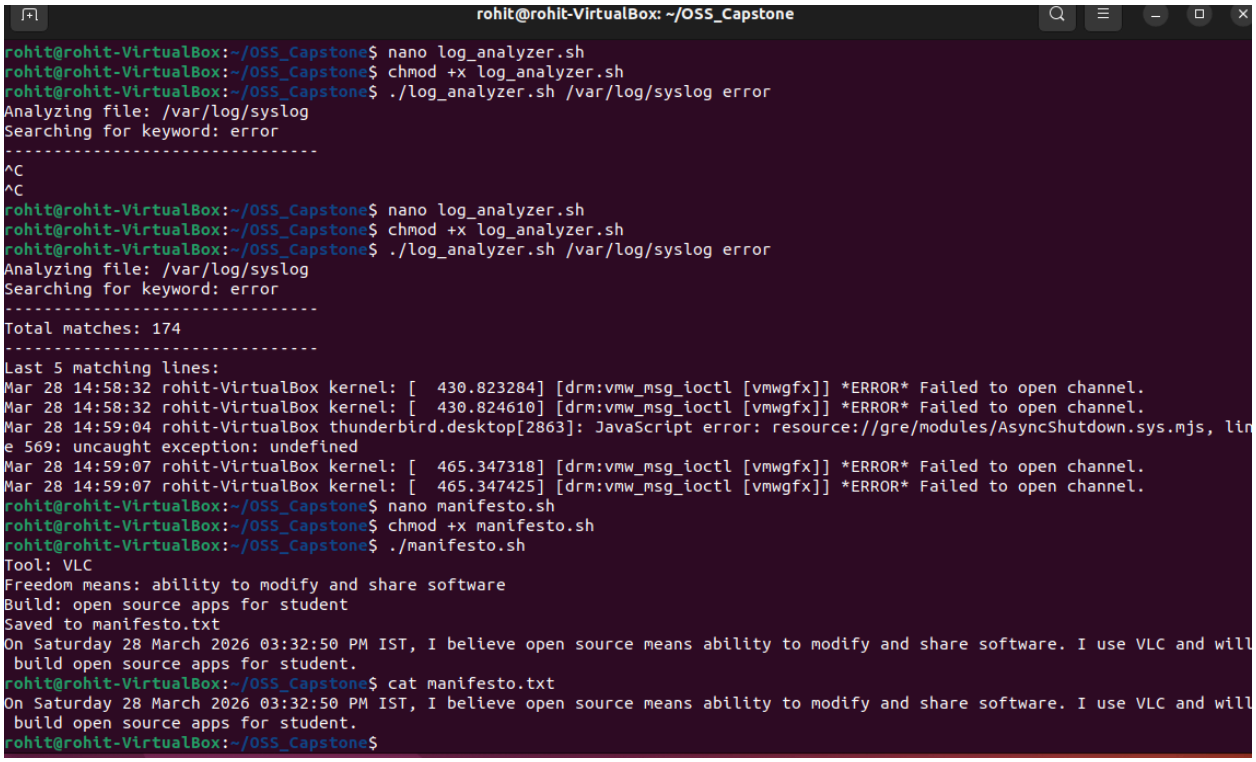
```
#!/bin/bash
# Script 4: Log File Analyzer
# Usage: ./log_analyzer.sh /var/log/messages
LOGFILE=$1
KEYWORD=${2:-"error"} # Default keyword is 'error'
COUNT=0
if [ ! -f "$LOGFILE" ]; then
echo "Error: File $LOGFILE not found."
exit 1
fi
while IFS= read -r LINE; do
if echo "$LINE" | grep -iq "$KEYWORD"; then
COUNT=$((COUNT + 1))
fi
done < "$LOGFILE"
echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
# TODO: Add a do-while style retry if the file is empty,
# and print the last 5 matching lines using tail + grep
```




the script automates the process of searching through log files for specific keywords such as "error" or "warning" while using a while loop to read the file line by line. It demonstrates positional parameters and string pattern matching with GREP to count occurrences of the term, making it an essential tool for troubleshooting system health and application performance

## 5. Open Source Manifesto Generator

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
echo "Answer three questions to generate your manifesto."
echo ""
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD
DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"
# TODO: Compose a paragraph using $TOOL, $FREEDOM, $BUILD
# and write it to $OUTPUT using echo and >>
echo "Manifesto saved to $OUTPUT"
cat $OUTPUT
```




this script is an interactive tool that uses read command to capture user input and store it in local variable to generate a first slide open Source Manager. It demonstrates string concatenation and file redirection by combining user responses with system data to create save a persistent text file, illustrating how scripts can bridge the gap between user interaction and file management